

權重是怎麼被改的

對應程式碼：[scratch/adamw.py](#)

分界線

```
model.backward(dlogits)    # 算出「該往哪修」 → 存進 dE、dW
opt.step()                 # 這一行才真的改動權重
```

反向傳播從不修改權重。這兩件事之間隔著一整張梯度表。

最直覺的版本：SGD

$$E \leftarrow E - lr \cdot g$$

「減掉梯度乘上學習率」。為什麼是減？梯度的定義是「往哪動 loss 會變大」，我們要 loss 變小，所以走反方向。這就是梯度下降的下降。

AdamW 做了什麼

```
t ← t + 1

p ← p - lr·wd·p                                ① decoupled weight decay (跟梯度無關)

m ← β1·m + (1-β1)·g                            ② 一階動量：梯度的移動平均
v ← β2·v + (1-β2)·g2                            ③ 二階動量：梯度平方的移動平均

m̂ ← m / (1-β1t)                                ④ bias correction
v̂ ← v / (1-β2t)

p ← p - lr · m̂/(√v̂ + ε)                        ⑤ 真正的更新
```

跟 SGD 的差別只在 ⑤： g 被換成了 $\hat{m}/(\sqrt{\hat{v}}+\epsilon)$ 。

關鍵性質：位移量幾乎與梯度大小無關

看 ⑤ 的分子分母：分子是梯度的平均，分母是梯度大小的平均。量級相同，相除之後大約落在 ± 1 ，於是每一步的位移 $\approx lr$ 。

第一步時這件事是精確成立的：

$$\begin{aligned}
 m_1 &= (1-\beta_1)g & \hat{m}_1 &= m_1/(1-\beta_1) = g \\
 v_1 &= (1-\beta_2)g^2 & \hat{v}_1 &= v_1/(1-\beta_2) = g^2 \\
 \hat{m}/\hat{v} &= g/|g| = \text{sign}(g) & \rightarrow & \text{位移恰好是 } \pm 1r
 \end{aligned}$$

追蹤檔裡可以驗證：

追蹤參數：embed.E (前 4 個數字)

梯度 g	+0.244449	-0.286912	-0.239161	+0.236112
$\hat{m}/(\hat{v}+\epsilon)$	+1.000000	-1.000000	-1.000000	+1.000000
位移 Δ	-0.000500	+0.000500	+0.000500	-0.000500
$1r$	0.000500			

四個維度的梯度大小不同，位移卻**一模一樣**。

推論：總位移有上限

跑 T 步之後，任何一個參數的總位移上限大約是 $T \times 1r$ ，不管它的梯度是大是小。

$$3000 \text{ 步} \times \text{平均 } 1r \ 2.75e-4 \approx 0.825$$

差別只在**方向**有沒有一致：

	位移	與初始向量的 $\cos$
高頻 token	接近上限	高（方向沒怎麼變）
中頻 token	接近上限	低（轉得最多）
從沒出現的 token	接近上限	中

位移量被 Adam 封頂，真正有差別的是「動得有沒有方向」。

高頻 token 出現在幾百萬個不同上下文，每個上下文想把它拉往不同方向，加起來互相抵消——所以它動得遠，但方向沒怎麼變。

decoupled weight decay

舊的 Adam 把 $wd \cdot p$ 加進梯度裡：

$$p \leftarrow p - lr \cdot (g + wd \cdot p)$$

問題是這樣一來 weight decay 也會被 $\hat{v}$ 正規化，強度變得跟梯度大小有關，很不直覺。

AdamW 把它拆出來獨立做（上面的 ①），強度固定是 $lr \cdot wd$ 。這就是 **W** 的由來。

一個實務上的坑：`torch.optim.AdamW` 預設 `weight_decay=0.01`，而且會套用到**所有**參數，包括 embedding 和 norm 的 gain。

生產級訓練通常把這兩類排除——因為很少拿到梯度的列會被 wd 慢慢磨向 0。這份程式用 `no_decay` 參數控制，預設就排除了：

```
AdamW(model.params(), lr=..., no_decay=('.g', '.E'))
```

梯度裁剪

```
opt.clip_grad_norm(1.0)
```

把所有梯度看成一個超長向量，如果它的長度超過門檻，**整體按比例縮小**。

注意是「整體」——各參數之間的相對比例不變。這跟逐個裁剪（clip by value）不同，後者會扭曲梯度的方向。

用途是防止偶爾出現的異常大梯度（壞資料、數值不穩）一步把模型炸掉。

learning rate schedule

```
lr = cosine_lr(step, total, base_lr, warmup=100)
```

- **warmup**：前 100 步從 0 線性升到 `base_lr`。一開始權重是亂數，梯度方向不可靠，這時候用大 lr 容易走歪。
- **cosine 衰減**：之後平滑降到 `base_lr × 0.1`。後期要小步微調，不然會在最低點附近震盪。

`traces/train_log.txt` 記了每一步的 lr，可以直接畫成曲線。